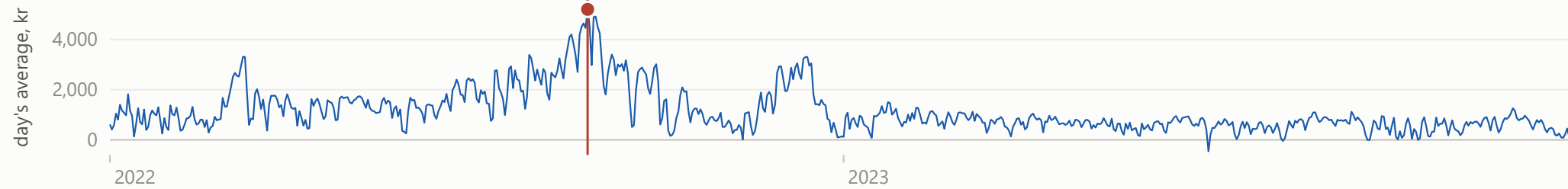


Electricity prices in West Denmark

Machine Learning in Finance · Session 10

Two years of prices

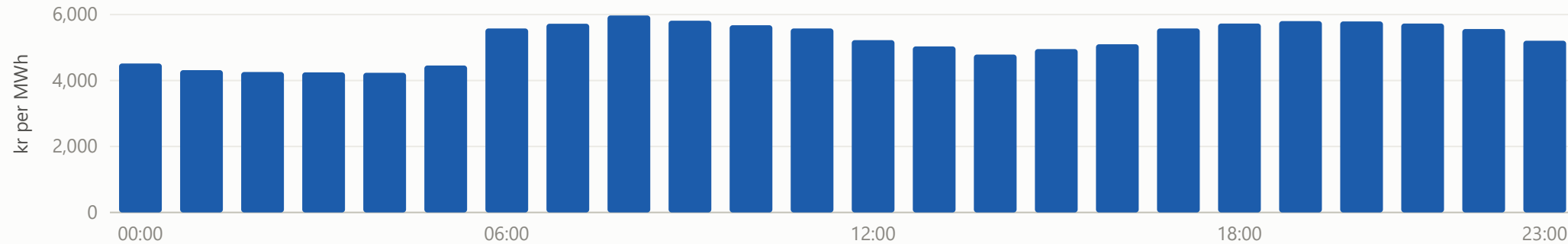
PLAY



the most expensive day

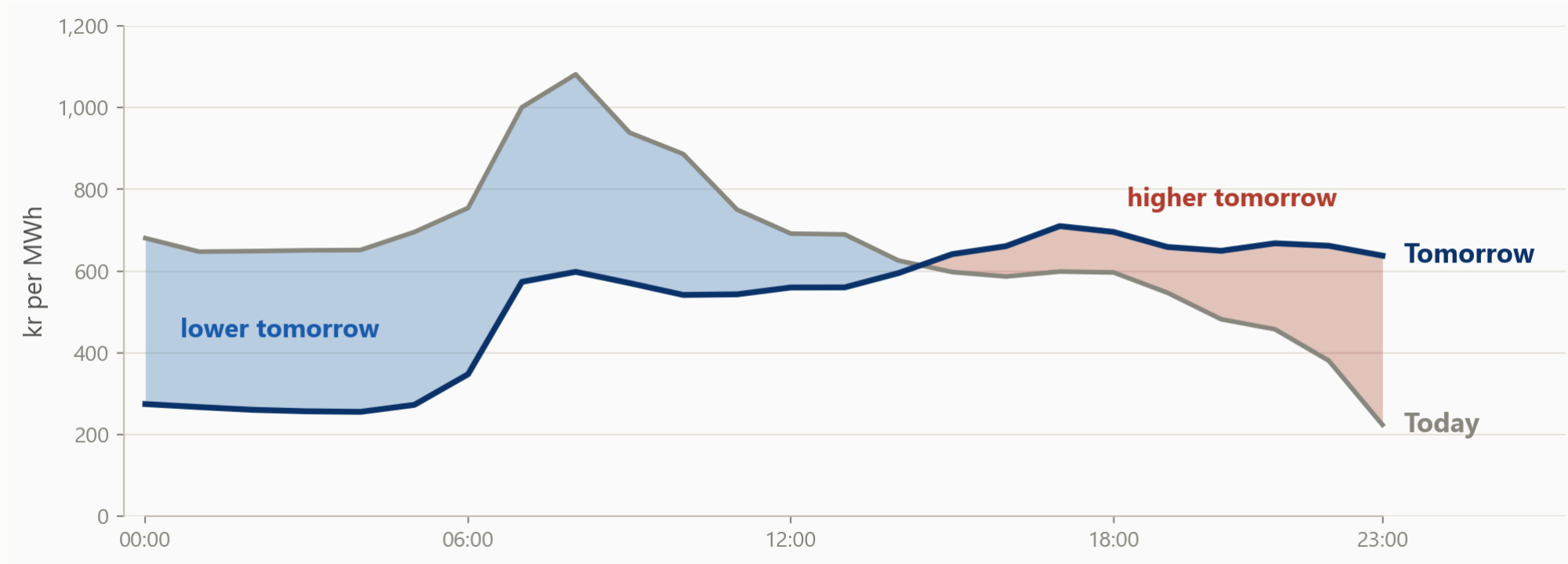
the cheapest day

Friday 26 August 2022 average 5,202 kr per MWh



Each point on the line is one day's average price in 2022 and 2023, the years the models learn from. Click, drag or step to see a day's 24 hours.

The question for 2024



Will tomorrow's price at this hour be higher than today's? In this example the answer is no until 14:00, and yes from 15:00.

Your forecasts for 2024

PLAY

tomorrow, at the hour	price yesterday, kr	price today, kr	wind forecast today, MW	wind forecast tomorrow, MW	how likely is tomorrow's price higher than today's?	
Tue 2 Jan, 13:00	120	330	1,382	3,447		not set
Wed 24 Apr, 19:00	1,224	918	721	514		not set
Thu 23 May, 12:00	205	153	2,666	506		not set
Fri 12 Jul, 20:00	873	1,375	497	668		not set
Sat 31 Aug, 10:00	688	733	724	600		not set
Sun 15 Sep, 09:00	867	497	1,047	241		not set
Thu 10 Oct, 18:00	1,074	914	980	3,030		not set
Mon 2 Dec, 17:00	1,021	820	2,047	1,691		not set

The models see the same numbers and the weekday. Your answers stay in this browser. [clear my answers](#)

Set how likely tomorrow's price is to be higher than today's, at each of these eight hours. Your answers are kept, and two models answer the same hours at the end.

The spot price on a household bill

578 kr

170 kWh in August 2022

109 kr

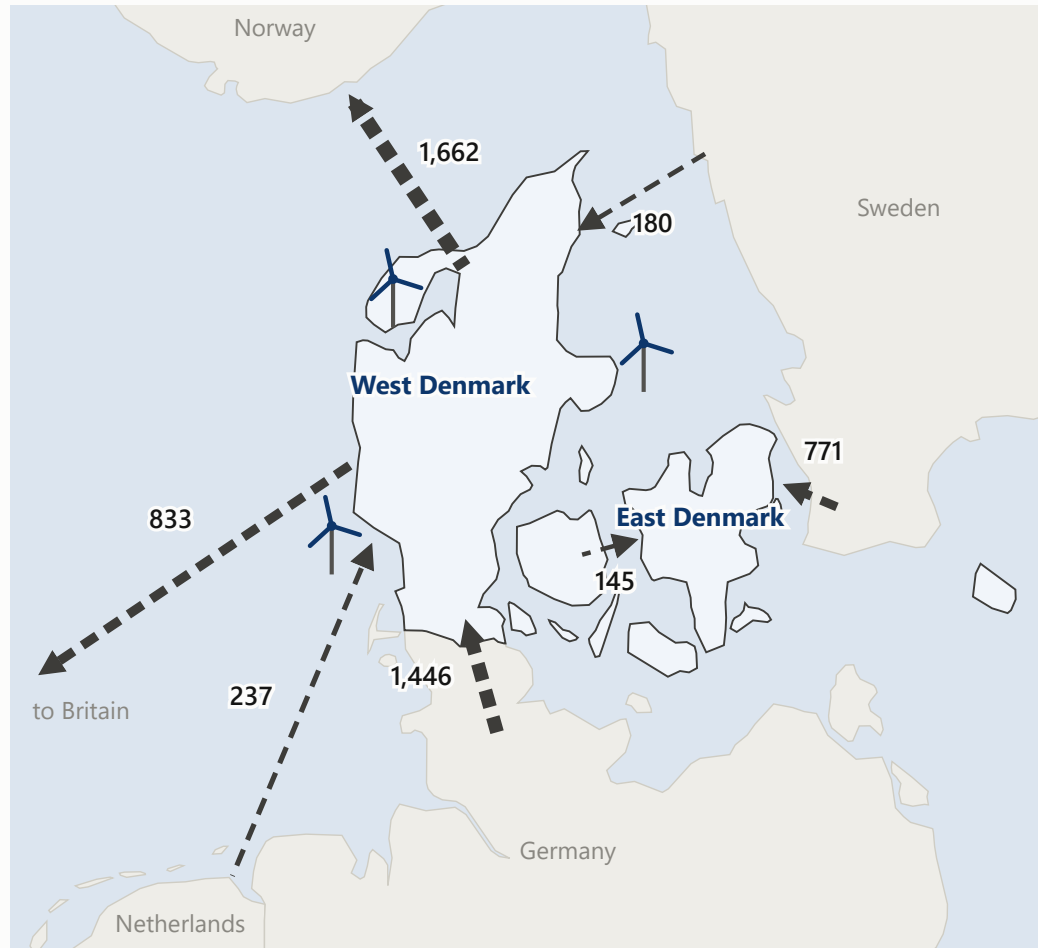
170 kWh in August 2023

93 kr

170 kWh in August 2024

On a spot-price contract a household pays each hour's price, plus tariffs and taxes. For 170 kWh, the spot part cost 578 kroner in August 2022 and 93 in August 2024.

Where the power flows PLAY



Windy: Monday 16 December

Still: Friday 4 October

00:00



play the day

West Denmark **1** kr per MWh · East Denmark **1** · wind forecast **3,529** MW



West Denmark through the day, kr per MWh



Arrows: power on each cable, MW, pointing the way it flows. Cables drawn schematically.

Denmark is two price areas, joined to five neighbours by cable. The auction sends power from cheaper areas to more expensive ones, as far as the cables allow.

Today, in six steps

Part 1	The market: the auction, the file, a price per year, prices below zero, a year hour by hour	pandas · matplotlib
Part 2	Why the price moves: the hour, the month, the wind, today against tomorrow, the S&P 500	groupby · pd.cut · a loop
Part 3	A first classifier: the label, a rule of thumb, the weekday as seven columns, today's move	shift · LogisticRegression
Part 4	What it is worth: a bet on the direction, the threshold and the cost in kroner	a function · a loop
Part 5	The crisis in the training rows: a label on the level, two training windows, the probabilities checked	Brier · calibration
Part 6	Similar hours: three classes, a big move either way, k-nearest neighbours, a feature lab, your forecasts revised and scored	abs · KNeighborsClassifier

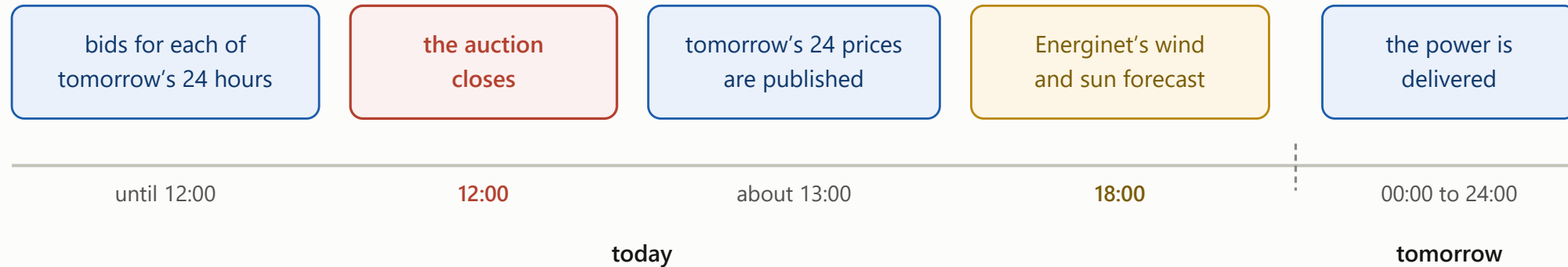
One pattern is new, **shift** inside **groupby**, and it is explained where it first appears. A slide marked **PLAY** has controls to click, slide or tick; a slide marked **YOUR TURN** has its answer on the slide after.

1 • The market

PART 1

How one auction sets 24 prices a day, the file, a price per year, prices below zero, and a year hour by hour.

The day-ahead auction



At noon today one auction sets all 24 of tomorrow's prices, and today's are already known. The forecast in the file comes out at 18:00 and stands in for a bidder's own.

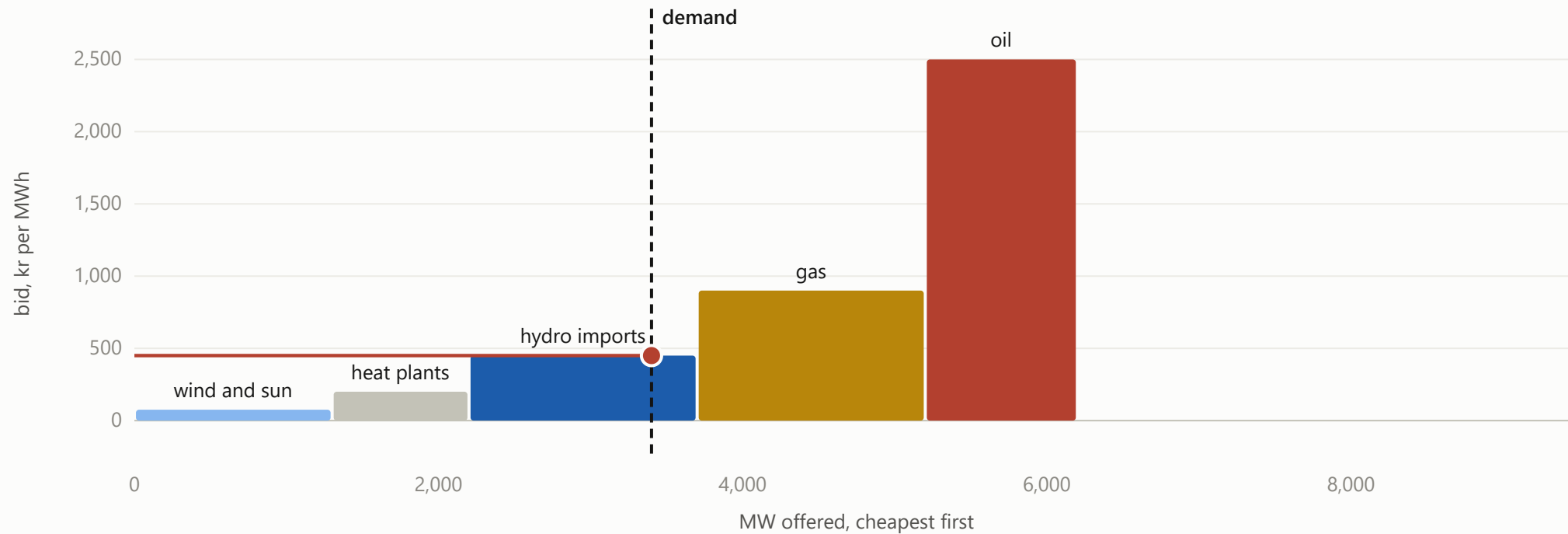
The merit order

PLAY

wind forecast 1,300 MW

demand 3,400 MW

Price **450** kr per MWh for every MWh sold, set by hydro imports



The offers are stacked from cheapest to most expensive, and the last one needed sets the price for all. Wind bids near zero; the blocks are stylised.

Reading the file

Python Code

```
power = pd.read_csv("data/power.csv", parse_dates=["time"])
power = power.set_index("time")

print(power.shape)
power.head()
```

(26133, 7)

		date	hour	weekday	weekend	price	wind	solar
time								
2022-01-01	00:00:00	2022-01-01	0	5	1	372.20	1623.2	0.0
2022-01-01	01:00:00	2022-01-01	1	5	1	307.35	1669.7	0.0
2022-01-01	02:00:00	2022-01-01	2	5	1	321.41	1622.7	0.0
2022-01-01	03:00:00	2022-01-01	3	5	1	338.06	1363.0	0.0
2022-01-01	04:00:00	2022-01-01	4	5	1	280.13	1163.0	0.0

The columns

price	the day-ahead price in West Denmark, kroner per MWh	wind	the forecast of wind output, MW
date	the day, as text: 2024-10-04	solar	the forecast of solar output, MW
hour	the clock hour, 0 to 23	time	the index: the hour, Danish time
weekday	0 for Monday to 6 for Sunday	weekend	1 on a Saturday or a Sunday

Seven days with no published forecast are left out. On the day the clocks go forward there are 23 hours, and on the day they go back the repeated hour is averaged.

A price per year

Python Code

```
prices = power['price']  
  
print(prices.groupby(power.index.year).mean())  
print(prices.max())
```

```
time  
2022    1629.726400  
2023     647.483481  
2024     530.549709  
Name: price, dtype: float64  
6982.42
```

Prices below zero

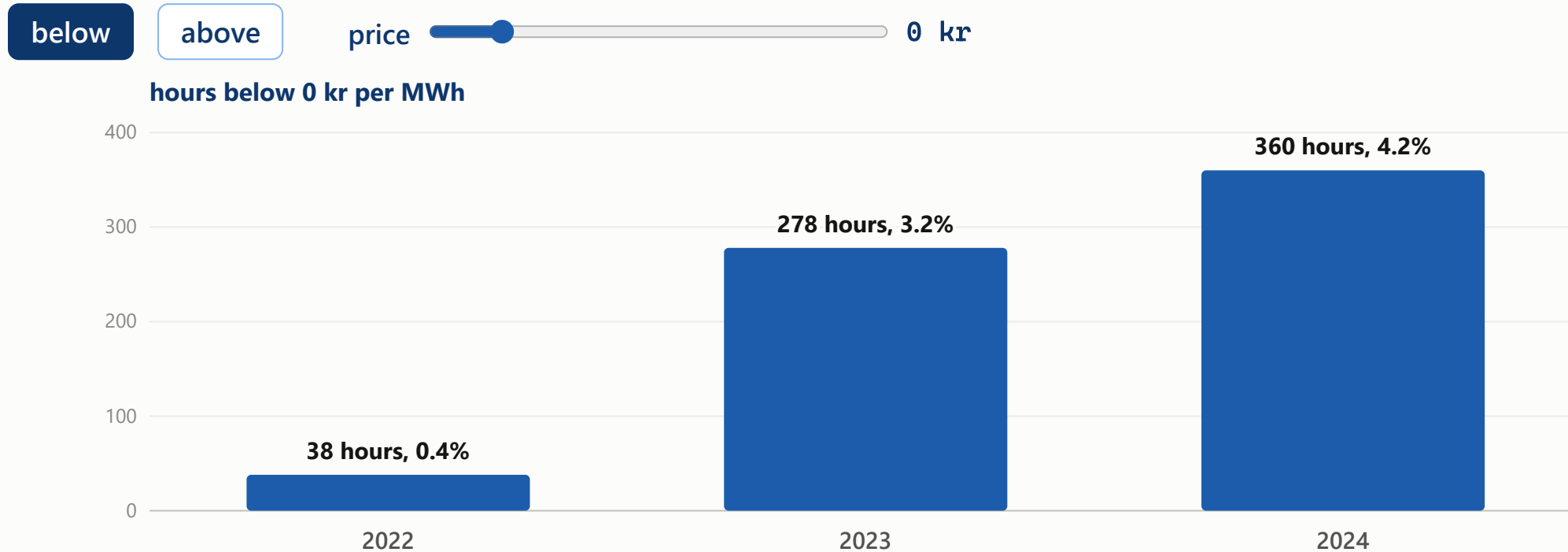
Python Code

```
below = power['price'] < 0

print(below.groupby(power.index.year).sum())
print(prices.min())
```

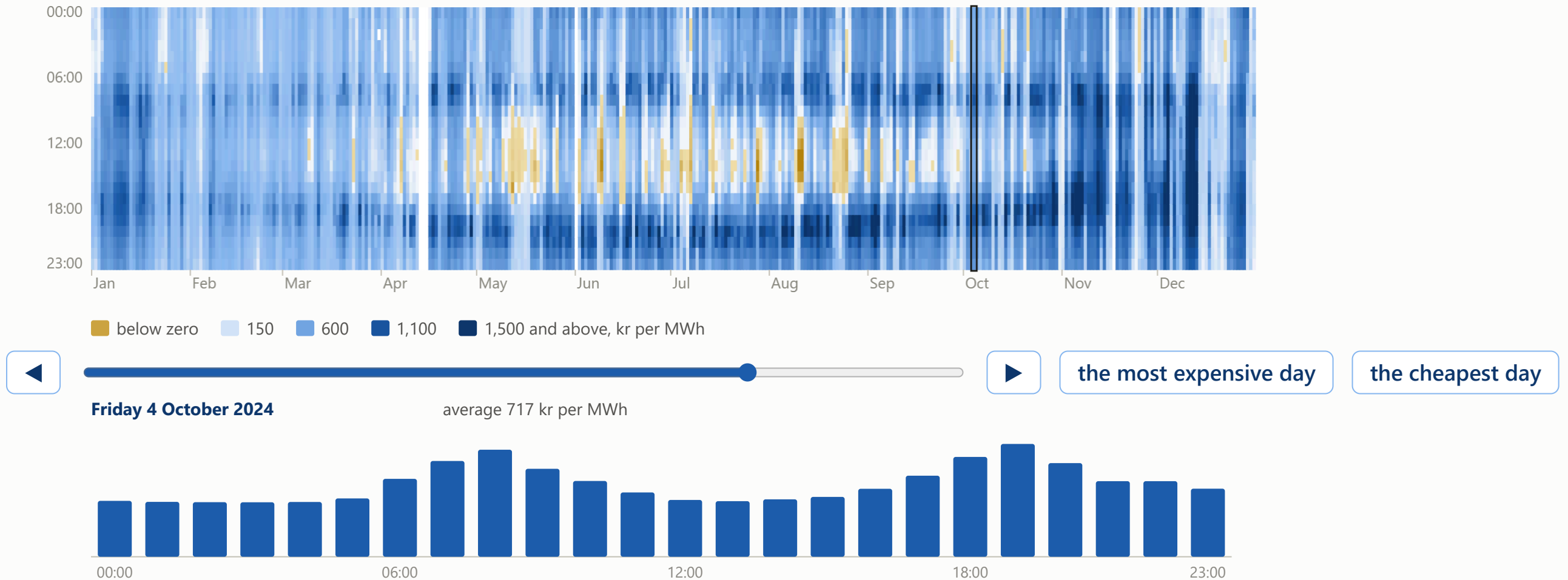
```
time
2022      38
2023     278
2024     360
Name: price, dtype: int64
-3277.39
```

Hours above or below a price PLAY



Slide the price and switch between below and above. Above 1,000 kroner there were 6,086 hours in 2022, and 525 in 2024.

2024, hour by hour PLAY



Each column is a day of 2024 and each row an hour. Step through the days or click one to draw its 24 prices; the dark bands are the daily peaks.

2 • Why the price moves

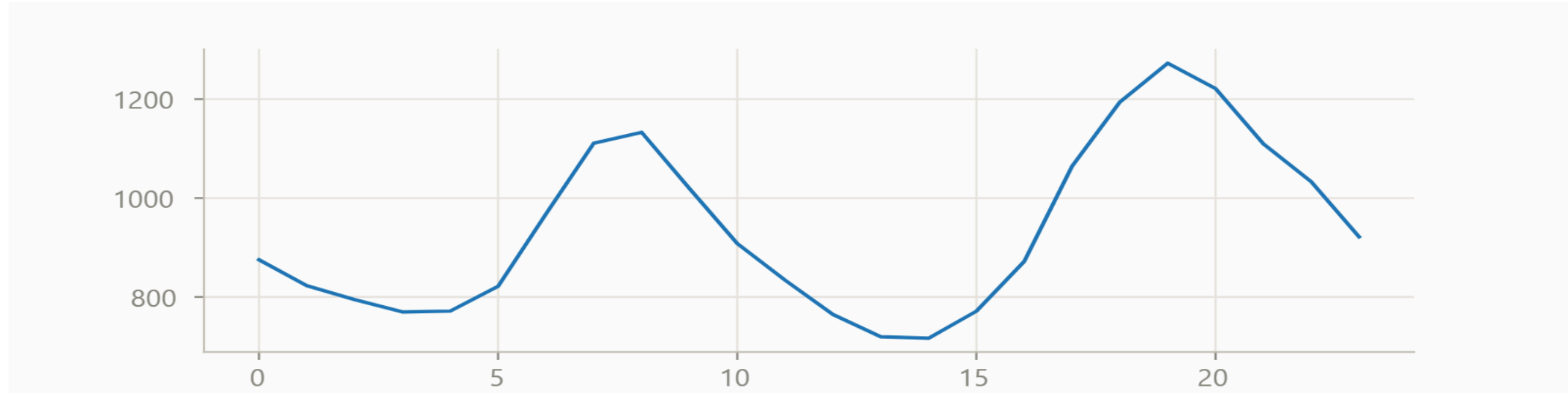
PART 2

The hour of the day, the month, the wind, today against tomorrow, and the size of the moves from one day to the next.

The shape of a day

Python Code

```
by_hour = power.groupby('hour')['price'].mean()  
fig, ax = plt.subplots(figsize=(8, 2.4))  
ax.plot(by_hour.index, by_hour.values)  
ax.set_xlabel("hour of the day")  
plt.show()
```



The shape, month by month

PLAY



January 2024

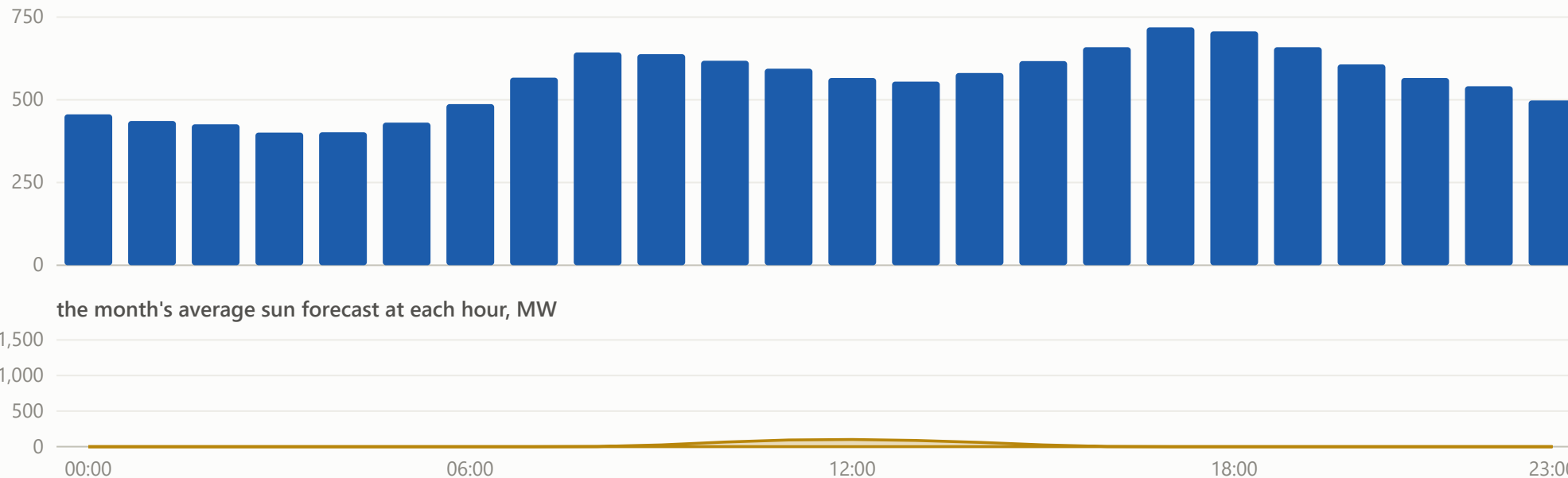


all days

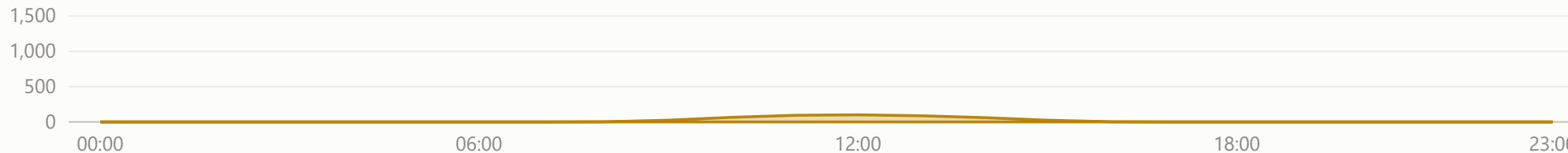
weekdays

weekends

the month's average price at each hour, kr per MWh



the month's average sun forecast at each hour, MW



Step through the months, and switch between weekdays and weekends. At weekends the morning peak almost disappears: 08:00 averages 777 kroner, against 1,274.

Wind, sun and the price PLAY

the wind forecast

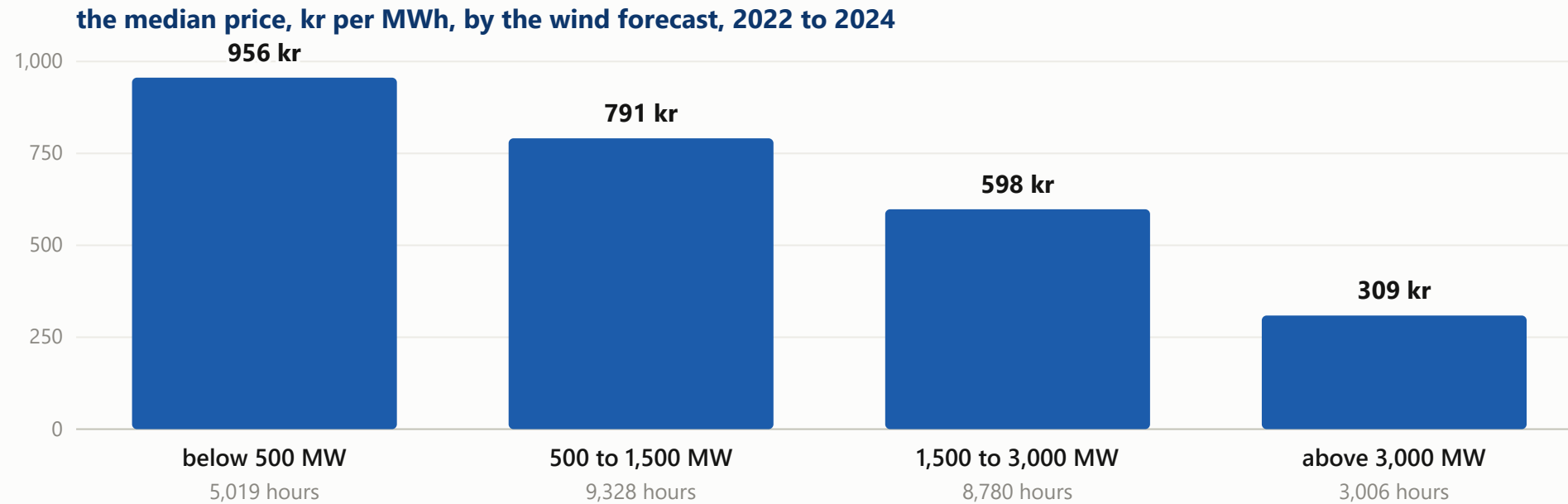
the sun forecast

all three years

2022

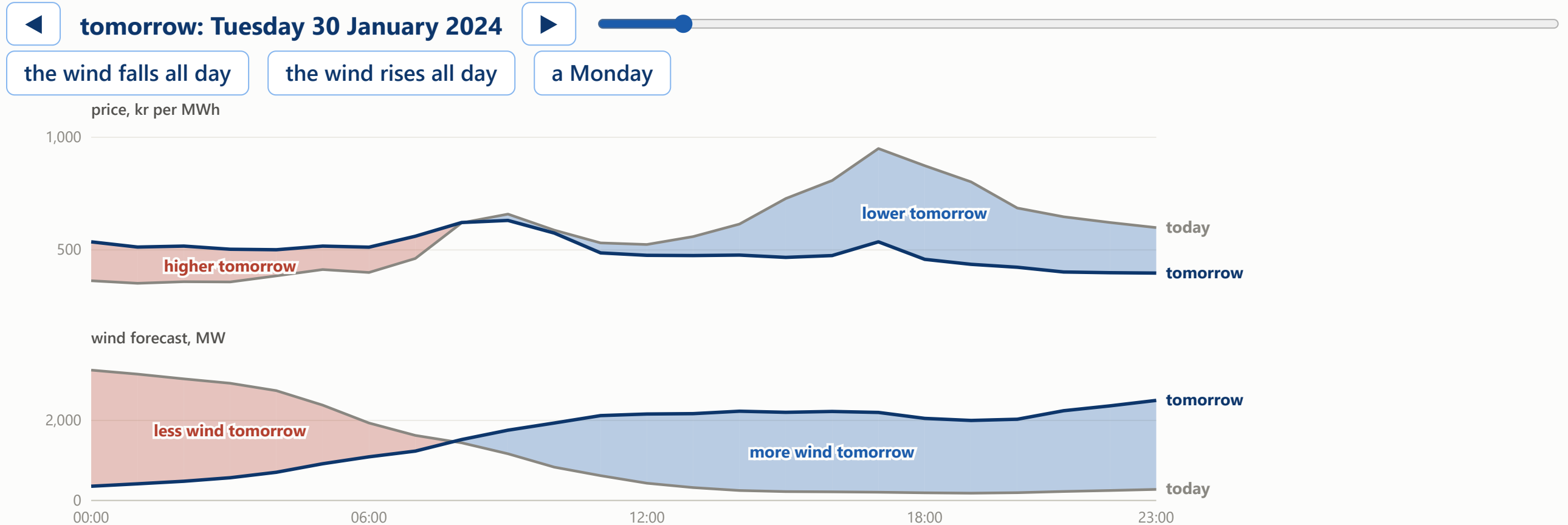
2023

2024



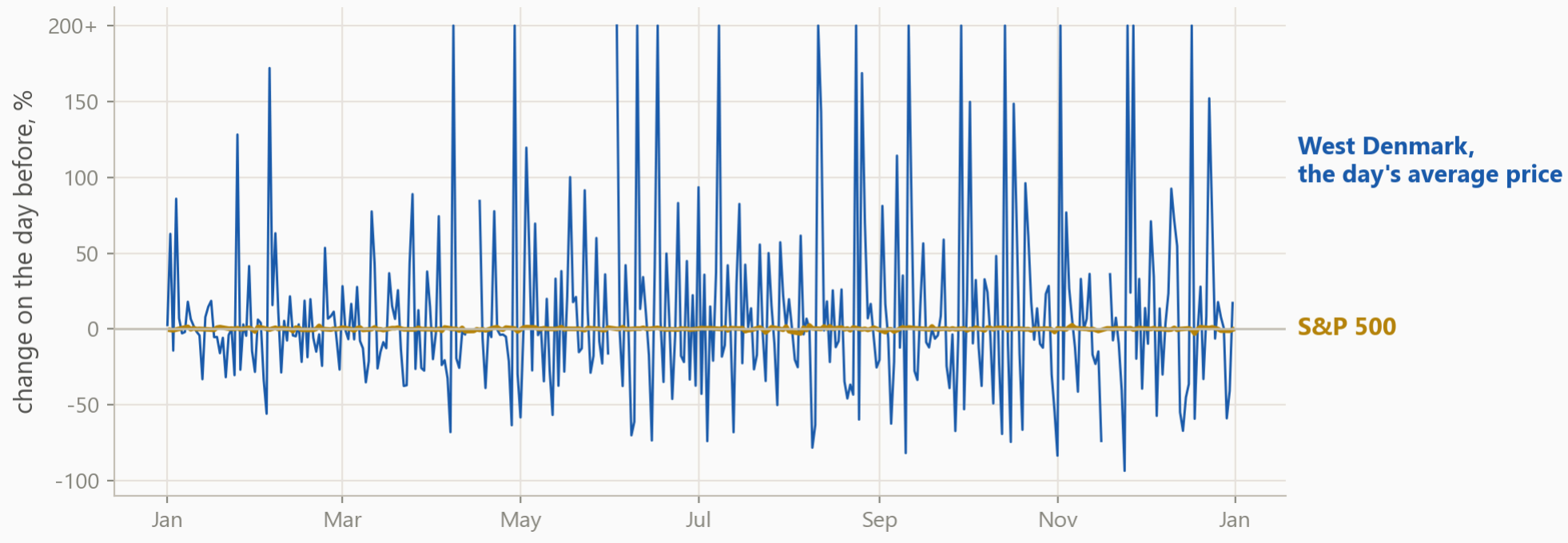
More wind lowers the median price in every year. The sun lowers it only when strong: above 1,000 MW the median is 348 kroner, against 722 with no sun.

Today and tomorrow PLAY



More wind tomorrow is blue, and so is a lower price, so the two panels mostly line up. On Monday 8 January they do not, and every hour rose.

Daily moves, power against the S&P 500



The day's average price moved more than 20 percent from the day before on 204 days of 2024. The S&P 500's largest daily move that year was 3.0 percent.

Your turn 1

Fill a dictionary with the median price at 03:00, 08:00, 13:00 and 18:00, then get all 24 medians in one line with **groupby**.

Python Code

```
medians = {}  
for h in [3, 8, 13, 18]:  
    medians[h] = ...  
    print(h, medians[h])  
  
print(power.groupby(...)[...].median())
```

Your turn 1: answer

```
medians = {}  
for h in [3, 8, 13, 18]:  
    medians[h] = power[power['hour'] == h]['price'].median()  
    print(h, medians[h])  
  
print(power.groupby('hour')['price'].median())
```

```
3 626.47  
8 844.57  
13 550.56  
18 895.19
```

The mask inside the loop keeps one hour at a time, and **groupby** does all 24 at once. The median at 18:00 is 43 percent above the median at 03:00.

3 • A first classifier

PART 3

The question from your forecasts as a label, a rule of thumb, the weekday as seven columns, and today's move.

The same hour, one day earlier

`groupby('hour')`

24 groups,
one per clock hour

08:00, one row per day	price	price_before	up
Sun 12 Nov	654	686	0
Mon 13 Nov	1,082	654	1
Tue 14 Nov	598	1,082	0
Wed 15 Nov	959	598	1

`shift(1)`

each price moves
down one row

Within each hour, `shift(1)` moves the prices down one row, so each row gets the day before's price. Returns within each ticker were built the same way.

The label

Python Code

```
power['price_before'] = power.groupby('hour')['price'].shift(1)
power['up'] = (power['price'] > power['price_before']).astype(int)

print(power.loc['2023-11-14 13:00':'2023-11-14 16:00', ['price_before', 'price', 'up']])
```

time	price_before	price	up
2023-11-14 13:00:00	689.86	560.14	0
2023-11-14 14:00:00	625.94	595.05	0
2023-11-14 15:00:00	597.67	641.52	1
2023-11-14 16:00:00	586.78	661.06	1

The wind change

Python Code

```
power['wind_change'] = power['wind'] - power.groupby('hour')['wind'].shift(1)

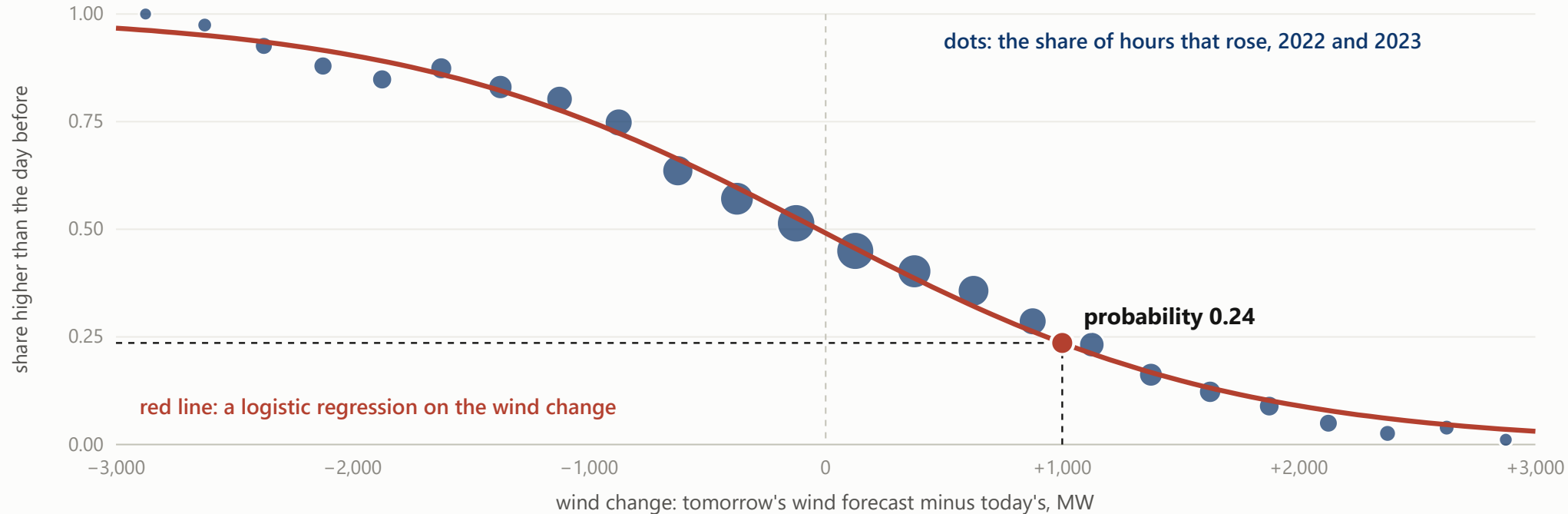
change_bands = pd.cut(power['wind_change'], [-np.inf, -1000, -300, 300, 1000, np.inf])
print(power.groupby(change_bands)['up'].mean())
```

```
wind_change
(-inf, -1000.0]    0.860768
(-1000.0, -300.0] 0.666534
(-300.0, 300.0]   0.489268
(300.0, 1000.0]   0.329131
(1000.0, inf]     0.124534
Name: up, dtype: float64
```

The logistic curve

PLAY

wind change



The dots are the share of hours that rose, and the red curve is a logistic regression on the wind change. Drag the slider to read its probability.

The weekday, as one number or seven columns

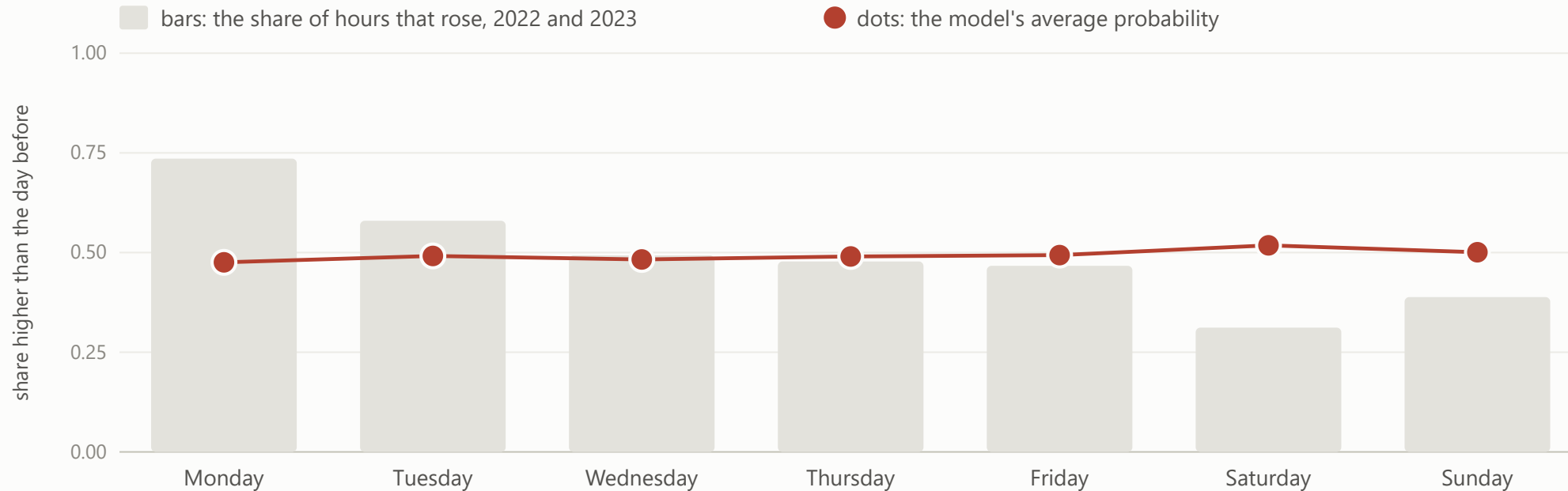
PLAY

without the weekday

the weekday as one number

the weekday as 7 columns

AUC on 2024 0.796



As one number, the weekday can only tilt the week one way. As seven columns, each day gets its own level, and the dots sit on the bars.

Weekday columns in code

Python Code

```
dummies = pd.get_dummies(power['weekday'], prefix='weekday', dtype=int)
days = []
for c in dummies:
    power[c] = dummies[c]
    days.append(c)
print(days)
```

```
['weekday_0', 'weekday_1', 'weekday_2', 'weekday_3', 'weekday_4', 'weekday_5', 'weekday_6']
```

`get_dummies` turns the weekday into seven columns of zeros and ones, and the loop copies each into the table and keeps its name. On every row exactly one of them is 1.

Today's move

Python Code

```
power['change_before'] = power['price_before'] - power.groupby('hour')['price'].shift(2)

move_bands = pd.cut(power['change_before'], [-np.inf, -200, -50, 50, 200, np.inf])
print(power.groupby(move_bands)['up'].mean())
```

```
change_before
(-inf, -200.0]    0.597029
(-200.0, -50.0]  0.523240
(-50.0, 50.0]    0.475918
(50.0, 200.0]    0.441127
(200.0, inf]     0.420427
Name: up, dtype: float64
```

The rows to learn from

Python Code

```
power = power.dropna()  
train = power.loc[:'2023']  
test = power.loc['2024']  
  
print(len(train), len(test))  
print(1 - test['up'].mean())
```

```
17422 8663  
0.5032898533995152
```

dropna removes the first two days, which have no day before. Calling no hour of 2024 up, the majority rule, is right 50.3 percent of the time.

A rule of thumb

Python Code

```
from sklearn.metrics import accuracy_score

rule = (test['wind_change'] < 0).astype(int)
print(accuracy_score(test['up'], rule))
```

0.7189195428835277

The rule says up whenever tomorrow's wind forecast is lower than today's. It is right 71.9 percent of the time in 2024, against 50.3 for the majority rule.

Logistic regression on the wind change

Python Code

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import roc_auc_score

model = Pipeline([('scale', StandardScaler()), ('logit', LogisticRegression())])
model.fit(train[['wind_change']], train['up'])
p = model.predict_proba(test[['wind_change']])[:, 1]
print(roc_auc_score(test['up'], p))
```

0.7956968628671697

With one column the model ranks the hours in the same order as the wind change itself: an AUC of 0.796. What it adds is a probability for every hour.

Adding columns one kind at a time

Python Code

```
ladder = {'+ weekday as a number': ['wind_change', 'weekday'],
          '+ weekday as 7 columns': ['wind_change'] + days,
          "+ today's move": ['wind_change'] + days + ['change_before']}
for name in ladder:
    model.fit(train[ladder[name]], train['up'])
    p = model.predict_proba(test[ladder[name]]][:, 1]
    print(name, round(roc_auc_score(test['up'], p), 4))
features = ladder["+ today's move"]
```

```
+ weekday as a number 0.8252
+ weekday as 7 columns 0.8299
+ today's move 0.8331
```

Each step adds one kind of column, and the AUC rises from 0.796 to 0.833. After the loop, `p` holds the probabilities of the last model, the one with all three.

Choose the columns PLAY

COLUMNS IN THE TABLE

- ☐ tomorrow's wind
- the wind forecast for the hour
- ☐ today's price
- the price at the same hour today
- ☐ weekday
- 0 for Monday to 6 for Sunday
- ☐ hour
- the clock hour, 0 to 23

COLUMNS BUILT FROM THEM

- ☒ wind change
- tomorrow's wind forecast minus today's
- ☐ today's move
- today's price minus yesterday's
- ☐ sun change
- tomorrow's sun forecast minus today's
- ☐ weekday as 7 columns
- one column of 0 and 1 per weekday

Tick a column to use it; the pandas line that builds it shows here.

AUC on the folds of 2022 and 2023, where the columns are chosen. Dashed: the model on the slides.

logistic regression

0.765



on 2024: 0.796

Your best on the folds: logistic regression 0.765. [clear](#)

Tick the columns the model may use; each tick shows what it adds on the folds of 2022 and 2023. The dashed line is the model on the slides.

C on the folds

Python Code

```
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV

folds = TimeSeriesSplit(n_splits=5)
grid = {'logit__C': [0.001, 0.01, 0.1, 1, 10]}
search = GridSearchCV(model, grid, cv=folds, scoring='roc_auc')
search.fit(train[features], train['up'])
print(search.best_params_, search.best_score_)
```

```
{'logit__C': 0.1} 0.8153848150821478
```

The folds move forward in time. Every C from 0.01 to 10 scores 0.815 on them, so the penalty barely matters here.

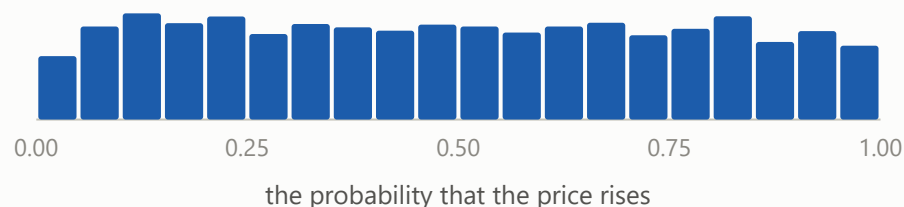
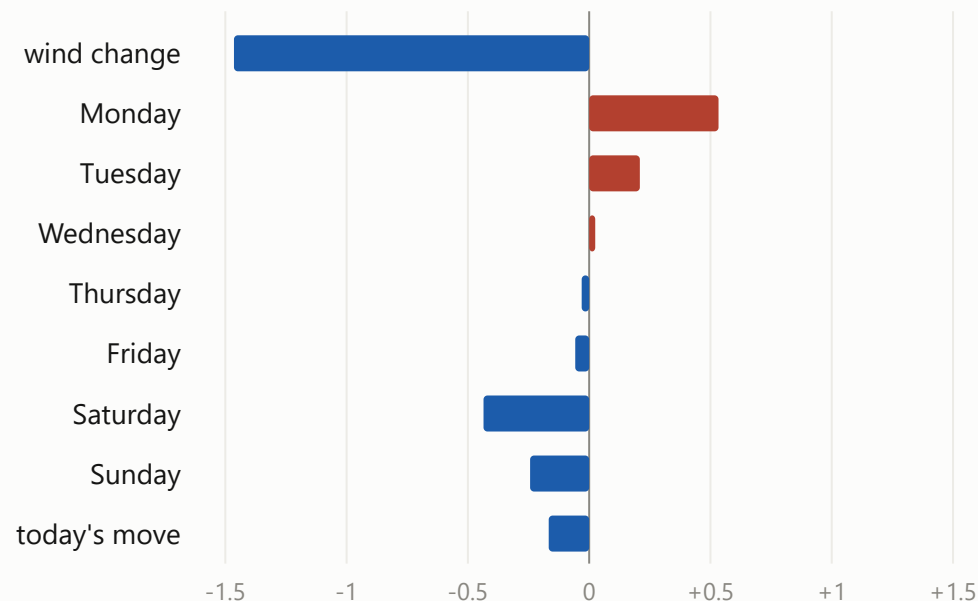
C, as a slider PLAY

C 10

AUC on the folds **0.815** · Brier score on 2024 **0.167**

the coefficients, on scaled columns

the model's probabilities for the hours of 2024



A small C pulls every coefficient toward zero and the probabilities toward one half. At 0.0001 the ranking barely changes, but the Brier score rises from 0.167 to 0.211.

The confusion matrix

Python Code

```
from sklearn.metrics import confusion_matrix, precision_score, recall_score

predicted = (p >= 0.5).astype(int)
print(confusion_matrix(test['up'], predicted))
print(precision_score(test['up'], predicted), recall_score(test['up'], predicted))
```

```
[[3274 1086]
 [1132 3171]]
0.7448907681465821 0.7369277248431327
```

At one half the model catches 3,171 of the 4,303 hours that rose, a recall of 0.737. When it says up, it is right 74.5 percent of the time.

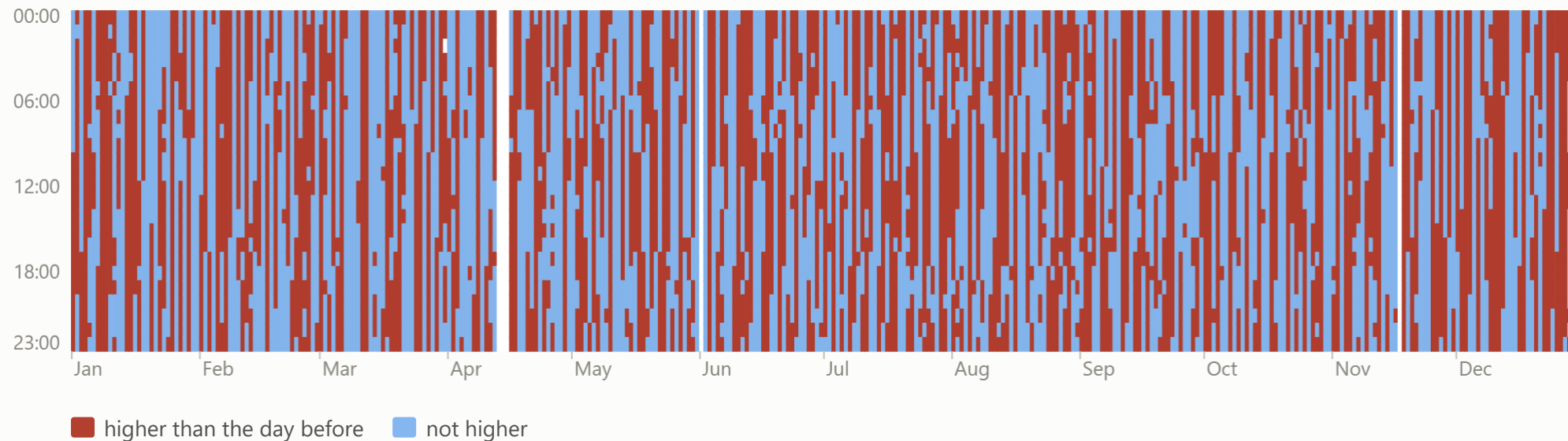
Predictions and mistakes, 2024

PLAY

what happened

what the model predicted

where it was wrong



On 143 of 2024's 361 days, at least 20 of the 24 hours moved the same way. The model was right on 85 percent of those hours, and on 68 percent of the others.

The same question for the S&P 500

74.4%

power, 2024: hours called
right by the model

50.3%

power, 2024: the majority
rule

50.4%

S&P 500, 2024: days called
right from yesterday's return
and the weekday

58.7%

S&P 500, 2024: always saying
up

A share can be bought today and sold tomorrow, so a rise anyone can predict is bought away before it happens. Power cannot be stored in bulk, so tomorrow's hour is a different good.

4 • What it is worth

PART 4

A bet on the direction of every hour, the kroner in each cell of the confusion matrix, and a threshold set by the cost of trading.

A bet on the direction

20:00 on Tuesday 14 November 2023: 482 kr today, 650 kr tomorrow, a move of +168 kr

long: a bet on higher	the move, minus the cost: $+168 - 50$	+118 kr
short: a bet on lower	minus the move and the cost: $-168 - 50$	-218 kr
no trade	nothing	0 kr

For each hour of 2024, bet 1 MWh on the direction: long when the probability is high, short when it is low. Each trade earns the move and pays a cost of 50 kroner.

The move in each hour

Python Code

```
move = test['price'] - test['price_before']
```

```
print(round(move.abs().sum()))
```

```
print(round(move.sum()))
```

1868568

514

Knowing every direction in advance, with no cost, would earn 1,868,568 kroner in 2024. Always going long earns 514, because the rises and falls almost cancel.

A function for the P&L

Python Code

```
def pnl(p, move, threshold, cost):  
    long = p >= threshold  
    short = p <= 1 - threshold  
    trades = long.sum() + short.sum()  
    return round(move[long].sum() - move[short].sum() - cost * trades)  
  
print(pnl(p, move, 0.5, 50))  
print(pnl(p, move, 0.6, 50))
```

879112

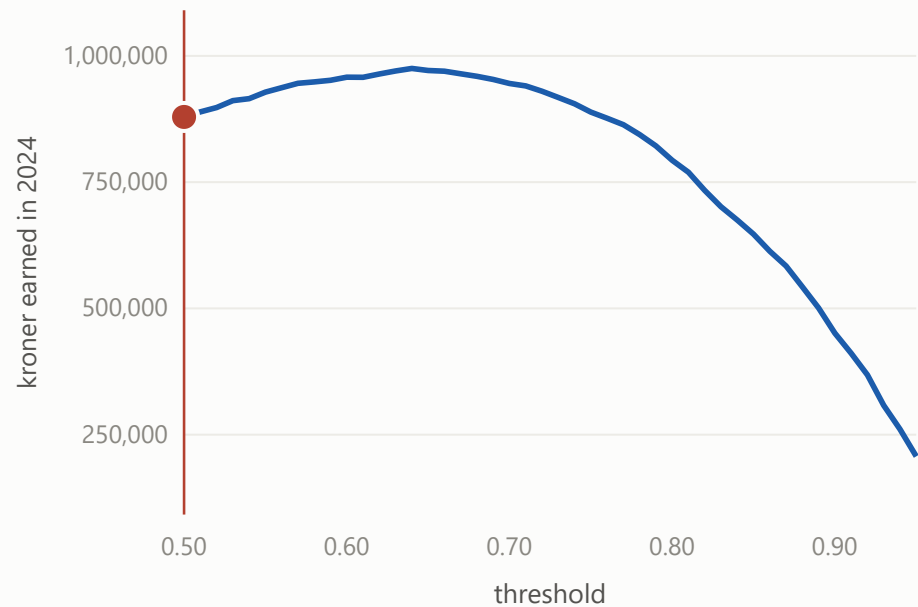
957809

At one half every hour is traded, and the model earns 879,112 kroner after costs. At 0.6 the hours near one half are left out, and it earns 957,809.

The threshold, in kroner

PLAY

threshold 0.50 879,112 kr from 8,663 of the 8,663 hours



	went up	did not go up
long: a bet on higher	3,171 +618,941 kr	1,086 -175,403 kr
short: a bet on lower	1,132 -213,650 kr	3,274 +649,224 kr
no trade	0 0 kr	0 0 kr

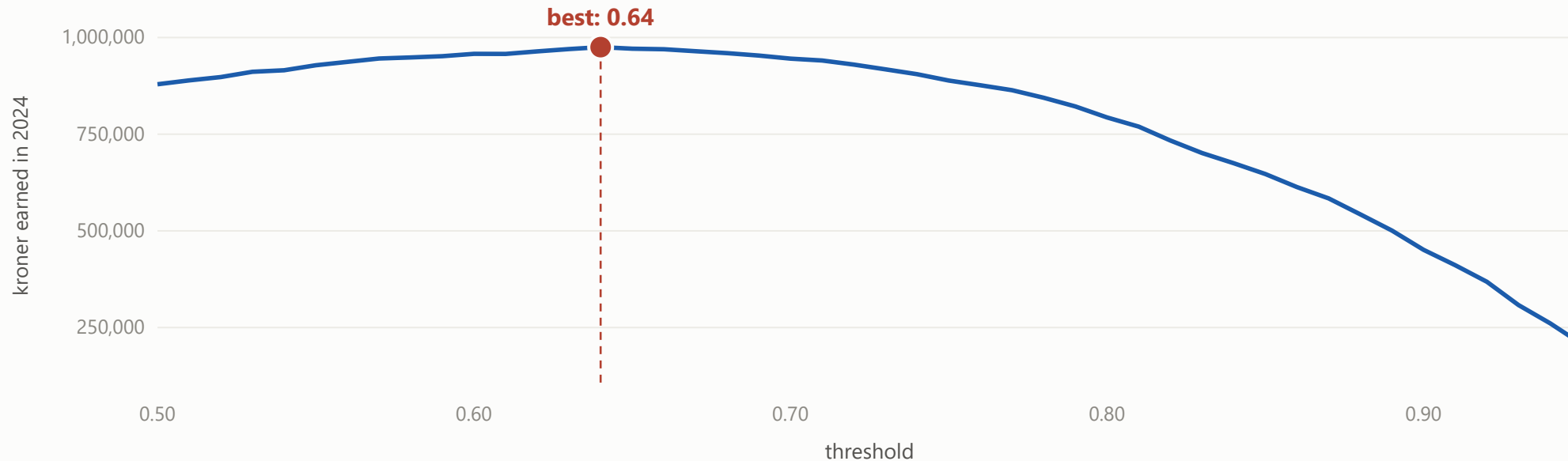
Each trade earns or loses the move and pays 50 kroner. At 0.64 the model trades 6,225 of the 8,663 hours and earns 975,120 kroner, the most of any threshold.

The cost of trading

PLAY

cost per MWh traded 50 kr

at the best threshold 975,120 kr; at 0.50 879,112 kr



The best threshold rises with the cost: 0.53 with no cost, 0.64 at 50 kroner and 0.77 at 150. At 150, one half earns 12,812 kroner and 0.77 earns 468,445.

Your turn 2

Choose the threshold on 2022 and 2023 instead, at a cost of 50. Fill **earned** with what **pnl** gives on the training rows at each threshold, and find the best.

Python Code

```
p_train = model.predict_proba(train[features])[:, 1]
move_train = train['price'] - train['price_before']

earned = {}
for t in [0.5, 0.55, 0.6, 0.65, 0.7]:
    earned[t] = ...
print(earned)
```

Your turn 2: answer

```
earned = {}  
for t in [0.5, 0.55, 0.6, 0.65, 0.7]:  
    earned[t] = pnl(p_train, move_train, t, 50)  
print(earned)  
# {0.5: 3276105, 0.55: 3354200, 0.6: 3340692, 0.65: 3284499, 0.7: 3157485}
```

On 2022 and 2023, 0.55 earns the most, and in 2024 it earns 928,247 kroner. That is 95 percent of the best threshold in hindsight, 0.64 with 975,120.

What the P&L leaves out

The cost

of 50 kroner per MWh is a round number for fees and the bid-ask spread, not a measured one.

The contract

that pays the change from one day to the next at one hour does not exist. The bet stands in for a buyer who can move use between days.

The size

of 1 MWh moves no price. A large bet placed in the auction would move the price against itself.

The forecast

in the file comes out at 18:00, after the auction, so the model saw a slightly better forecast than a bidder has.

5 • The crisis in the training rows

PART 5

A second label on the level of the price, an hour above 1,000 kroner, fitted with and without 2022, and its probabilities checked against 2024.

Prices above 1,000 kroner

Python Code

```
power['above_1000'] = (power['price'] > 1000).astype(int)

print(power['above_1000'].groupby(power.index.year).mean())
```

```
time
2022    0.700242
2023    0.132456
2024    0.060603
Name: above_1000, dtype: float64
```

In 2022, 70 percent of the hours cost more than 1,000 kroner, and in 2024, 6 percent. The share that rose from the day before stayed near half in every year.

Two training windows

Python Code

```
levels = ['wind', 'solar'] + days
crisis = Pipeline([('scale', StandardScaler()), ('logit', LogisticRegression())])
crisis.fit(power.loc[:'2023', levels], power.loc[:'2023', 'above_1000'])
calm = Pipeline([('scale', StandardScaler()), ('logit', LogisticRegression())])
calm.fit(power.loc['2023', levels], power.loc['2023', 'above_1000'])

print(crisis.predict_proba(test[levels])[:, 1].mean())
print(calm.predict_proba(test[levels])[:, 1].mean())
```

0.4049395621191426

0.12813970921197246

With 2022 and 2023 the model expects 40 percent of 2024's hours above 1,000 kroner, and with 2023 alone 13; the share was 6.

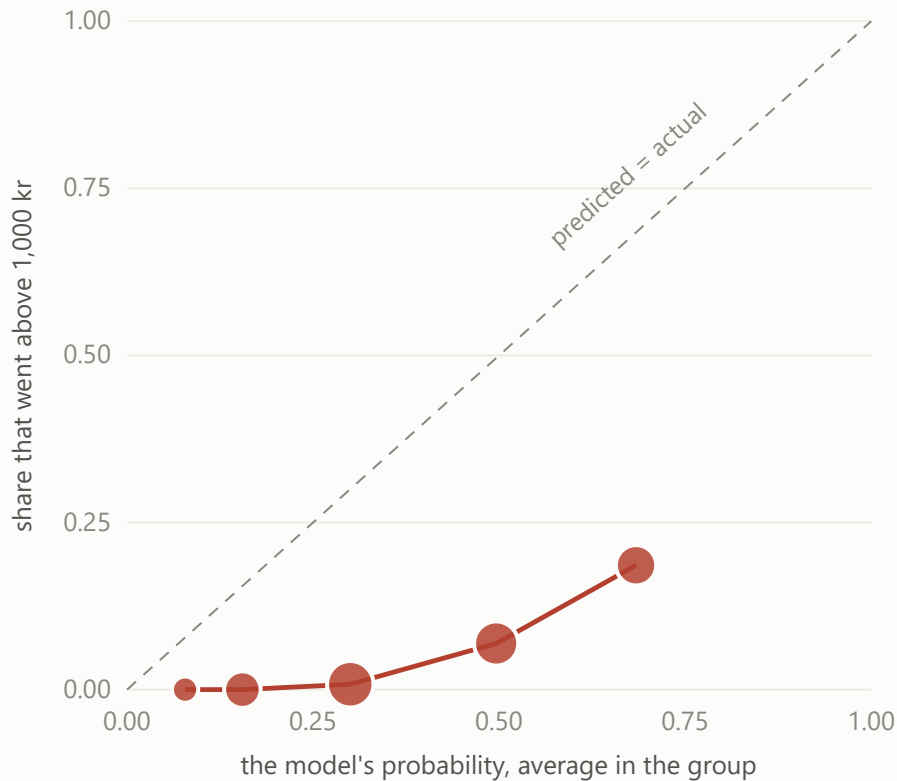
Predicted against actual

PLAY

trained on 2022 and 2023

trained on 2023 only

trained on 2022 only



hours above 1,000 kr in the training years	41.6%
the model's average probability for 2024	40.5%
hours above 1,000 kr in 2024	6.1%

Each dot is a group of 2024’s hours. With 2022 in training, a probability near 0.3 meant 0.8 percent; with 2023 alone, 15 percent.

AUC against the Brier score

Python Code

```
from sklearn.metrics import brier_score_loss

happened = power.loc['2024', 'above_1000']
for m in [crisis, calm]:
    q = m.predict_proba(test[levels][:, 1])
    print(roc_auc_score(happened, q), brier_score_loss(happened, q))
```

```
0.8392168427951174 0.18680001793096432
0.8384432819576589 0.06217430615306306
```

Both models rank 2024's hours equally well, with AUCs of 0.839 and 0.838. Only the Brier score shows the difference: 0.187 against 0.062.

Handling a changed market

A recent window

2023 alone forecast 2024 better than 2022 and 2023 together: a Brier score of 0.062 against 0.187.

A label that travels

The direction model, trained on the same years, gave 2024 an average probability of 0.495; the share that rose was 0.497.

Watch the average

Compare the mean probability with the share that happened, month by month, and retrain when the gap grows.

6 • Similar hours

PART 6

Three classes, a big move either way, the k most similar hours of the past, a feature lab, and your forecasts revised and scored.

Three classes

Python Code

```
change = power['price'] - power['price_before']
power['move'] = pd.cut(change, [-np.inf, -50, 50, np.inf], labels=['lower', 'about the same', 'higher'])
three = Pipeline([('scale', StandardScaler()), ('logit', LogisticRegression())])
three.fit(power.loc[:, '2023', features], power.loc[:, '2023', 'move'])
predicted_move = three.predict(test[features])
print(accuracy_score(power.loc[:, '2024', 'move'], predicted_move))
```

0.6059101927738658

`pd.cut` makes three classes: more than 50 kroner down, less than 50 either way, and more than 50 up. The majority rule scores 0.381 and the model 0.606.

The three-class confusion matrix

Python Code

```
order = ['lower', 'about the same', 'higher']  
actual = power.loc['2024', 'move']  
  
print(confusion_matrix(actual, predicted_move, labels=order))
```

```
[[2643    0   659]  
 [1127    0   979]  
 [ 649    0 2606]]
```

Rows are what happened, from lower to higher. The model never predicts about the same: in no hour of 2024 is it the most likely of the three.

A big move either way

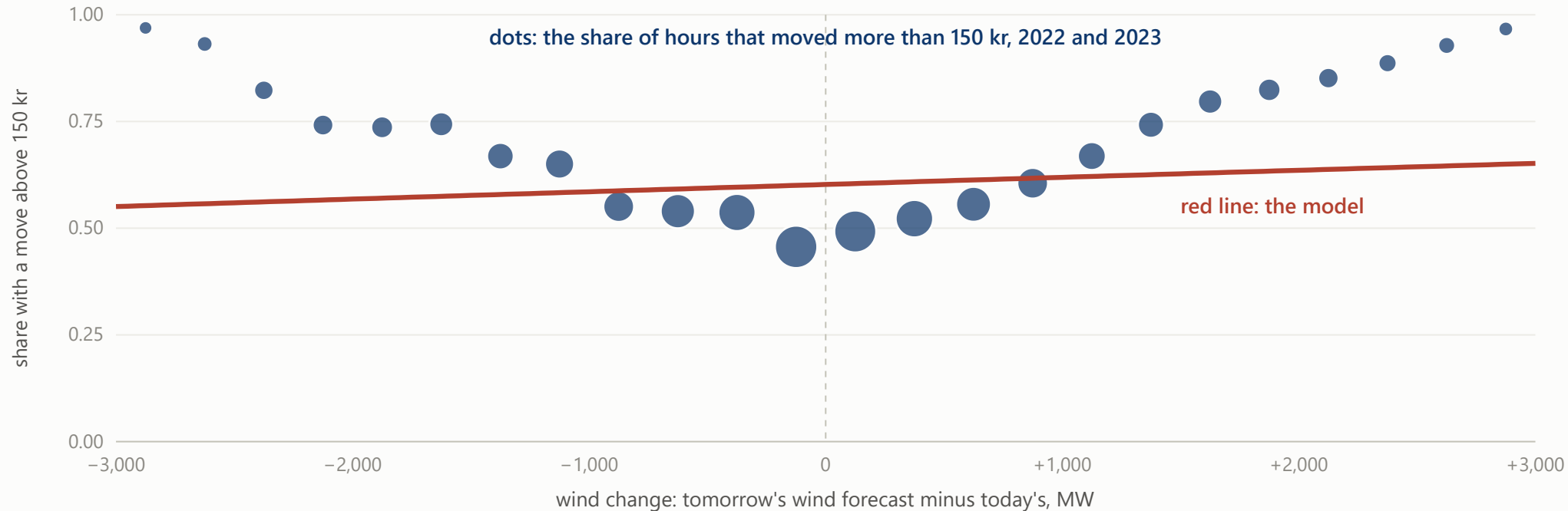
PLAY

logistic regression, the wind change

logistic regression, the size of the wind change

k-NN, the wind change

AUC on 2024 0.522



A move of more than 150 kroner is most likely when the wind forecast changes a lot, either way. A logistic regression on the wind change can only slope one way.

The size of the wind change

Python Code

```
power['big'] = (change.abs() > 150).astype(int)
power['size_wind_change'] = power['wind_change'].abs()

big_model = Pipeline([('scale', StandardScaler()), ('logit', LogisticRegression())])
for cols in [features, features + ['size_wind_change']]:
    big_model.fit(power.loc[:, '2023', cols], power.loc[:, '2023', 'big'])
    q = big_model.predict_proba(power.loc[:, '2024', cols])[:, 1]
    print(roc_auc_score(power.loc[:, '2024', 'big'], q))
```

0.5586539927493552

0.669963318820275

abs builds the size of the wind change, a shape a linear model cannot make from the change itself. The AUC for a big move rises from 0.559 to 0.670.

The k nearest hours

PLAY

wind change +1,000 MW

weekday Wednesday

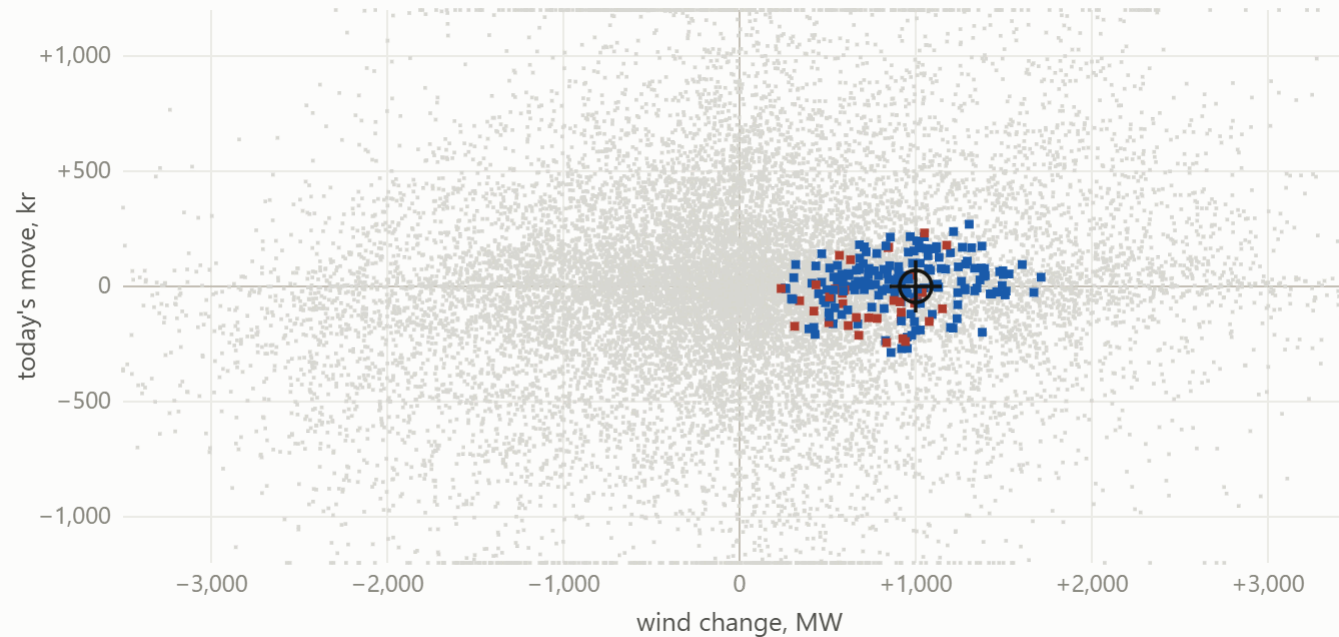
today's move 0 kr

hour 18:00

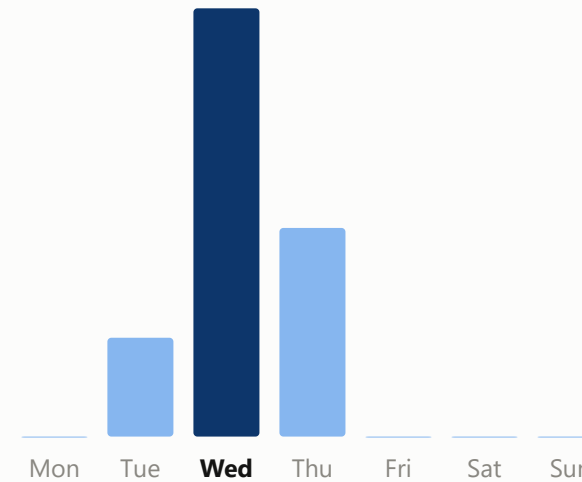
neighbours, k 201

☒ scale the columns first

Of the **201** most similar hours of 2022 and 2023, **36** went up (red), so k-NN says **0.18**.



the neighbours, by weekday



k-NN finds the k hours of 2022 and 2023 most like this one and reports the share that rose. Untick the scaler, and the weekday stops counting.

k-NN in code

Python Code

```
from sklearn.neighbors import KNeighborsClassifier

near = ['wind_change', 'change_before', 'weekday', 'hour']
knn = Pipeline([('scale', StandardScaler()), ('knn', KNeighborsClassifier(n_neighbors=201))])
knn.fit(train[near], train['up'])
print(roc_auc_score(test['up'], knn.predict_proba(test[near])[:, 1]))
```

0.8442208817402836

k-NN needs no weekday columns: the nearest hours to a Monday are Mondays. With k of 201, chosen on the folds, it scores 0.844 against the logistic regression's 0.833.

k-NN without the scaler

Python Code

```
bare = KNeighborsClassifier(n_neighbors=201)
bare.fit(train[near], train['up'])

print(roc_auc_score(test['up'], bare.predict_proba(test[near])[:, 1]))
print(train[near].std())
```

0.7973458884030132

wind_change 1122.558638

change_before 501.620005

weekday 1.998190

hour 6.922034

dtype: float64

The feature lab

PLAY

predict

tomorrow higher than today

a move of more than 150 kr either way

COLUMNS IN THE TABLE

- ☐ tomorrow's wind the wind forecast for the hour
- ☐ today's price the price at the same hour today
- ☐ weekday 0 for Monday to 6 for Sunday
- ☐ hour the clock hour, 0 to 23

COLUMNS BUILT FROM THEM

- ☒ wind change tomorrow's wind forecast minus today's
- ☐ today's move today's price minus yesterday's
- ☐ sun change tomorrow's sun forecast minus today's
- ☐ weekday as 7 columns one column of 0 and 1 per weekday
- ☐ size of the wind change the wind change without its sign

Tick a column to use it; the pandas line that builds it shows here.

AUC on the folds of 2022 and 2023, where the columns are chosen. Dashed: the model on the slides.

logistic regression

0.765

on 2024: 0.796

k-NN, k = 201

0.755

on 2024: 0.790

Your best on the folds: logistic regression **0.765**, k-NN, k = 201 **0.755**. [clear](#)

The same columns for k-NN, and for a big move either way. The dashed lines are the models on the slides, and they can be beaten.

Your forecasts, revised

PLAY

tomorrow, at the hour	wind change, MW	today's move, kr	your first answer	your answer now	
Tue 2 Jan, 13:00	+2,065	+210	not set		unchanged
Wed 24 Apr, 19:00	-207	-306	not set		unchanged
Thu 23 May, 12:00	-2,160	-52	not set		unchanged
Fri 12 Jul, 20:00	+171	+502	not set		unchanged
Sat 31 Aug, 10:00	-124	+45	not set		unchanged
Sun 15 Sep, 09:00	-806	-370	not set		unchanged
Thu 10 Oct, 18:00	+2,050	-160	not set		unchanged
Mon 2 Dec, 17:00	-356	-201	not set		unchanged

An answer you leave alone stays as it was.

[back to my first answers](#)

You have now seen what moves the price: the wind change, the weekday and today's move. Change any forecast you like; your first answers are kept.

The scoreboard

tomorrow, at the hour	what happened	your first answer	your answer now	logistic regression	k-NN
Tue 2 Jan, 13:00	higher: 330 → 588 kr	50%	50%	10%	10%
Wed 24 Apr, 19:00	higher: 918 → 1,009 kr	50%	50%	60%	61%
Thu 23 May, 12:00	higher: 153 → 392 kr	50%	50%	94%	89%
Fri 12 Jul, 20:00	not higher: 1,375 → 683 kr	50%	50%	36%	40%
Sat 31 Aug, 10:00	not higher: 733 → 448 kr	50%	50%	25%	16%
Sun 15 Sep, 09:00	not higher: 497 → 403 kr	50%	50%	61%	39%
Thu 10 Oct, 18:00	not higher: 914 → 294 kr	50%	50%	6%	6%
Mon 2 Dec, 17:00	higher: 820 → 1,097 kr	50%	50%	88%	89%
Brier score, average over the 8 hours		0.250	0.250	0.194	0.166
log loss, average over the 8 hours		0.693	0.693	0.590	0.532

Shaded: the closest forecast on each hour, and the lowest score. Answers you did not set count as 50 percent. Saying 50 percent every time scores 0.250 and 0.693.

Both scores are losses, so lower is better. The log loss punishes a confident wrong answer hardest, as on 2 January.

Everything used today

Looking at a market

`read_csv` with `parse_dates`, `set_index`, `.loc` by date, `groupby` by year and by hour, `pd.cut` into bands, a bar chart and two lines.

A label from the day before

`shift(1)` inside `groupby('hour')` puts the same hour of the day before on every row. About half the hours rose, in every year.

The threshold and the cost

The best threshold rose with the cost. Chosen on 2022 and 2023, 0.55 earned 928,247 kroner in 2024 at 50 kroner a trade.

A shape a line cannot make

The size of the wind change took a big move's AUC from 0.559 to 0.670, and k-NN found the shape by itself.

Loops, dictionaries, functions

A dictionary of medians filled by a loop, a loop that adds the weekday columns, a ladder of models in a dictionary, and `pn1` on two sets of rows.

Columns built from others

The wind change gave an AUC of 0.796, the weekday as seven columns 0.830, and today's move 0.833.

Training rows from another market

Trained through 2022, the model expected 40 percent of 2024's hours above 1,000 kroner. The share was 6 percent.

k-NN and the scoreboard

The weekday and the hour as plain numbers, scaled: an AUC of 0.844. On your eight hours k-NN scored 0.166 and logistic regression 0.194.

Where this goes next

before	logistic regression, thresholds and costs, calibration, three classes and k-NN	classification
now	one market from the auction to the P&L, with every tool so far	a recap
next	trees and ensembles, for numbers and for labels	the same workflow

The label, the split, the folds, the threshold and the scores stay exactly as they were today. Only the model inside them changes.

Before the next session

- There are no exercises and no case this time. Open the deck at home: beat the models on the slides in the feature lab, and change the live cells, with 2023 as the test year or another k.
- Finish any earlier exercise set you skipped. The loops, functions and masks in the early sets carried every step today.
- Read the case notebooks for Parts 8 and 9 again in one sitting, as one classification investigation.

Stuck? Ask a friend, ask an AI for a hint rather than an answer, or email jobo@econ.au.dk.